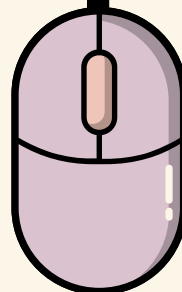
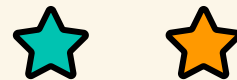


Davante

The Clean Code & Automation Cycle

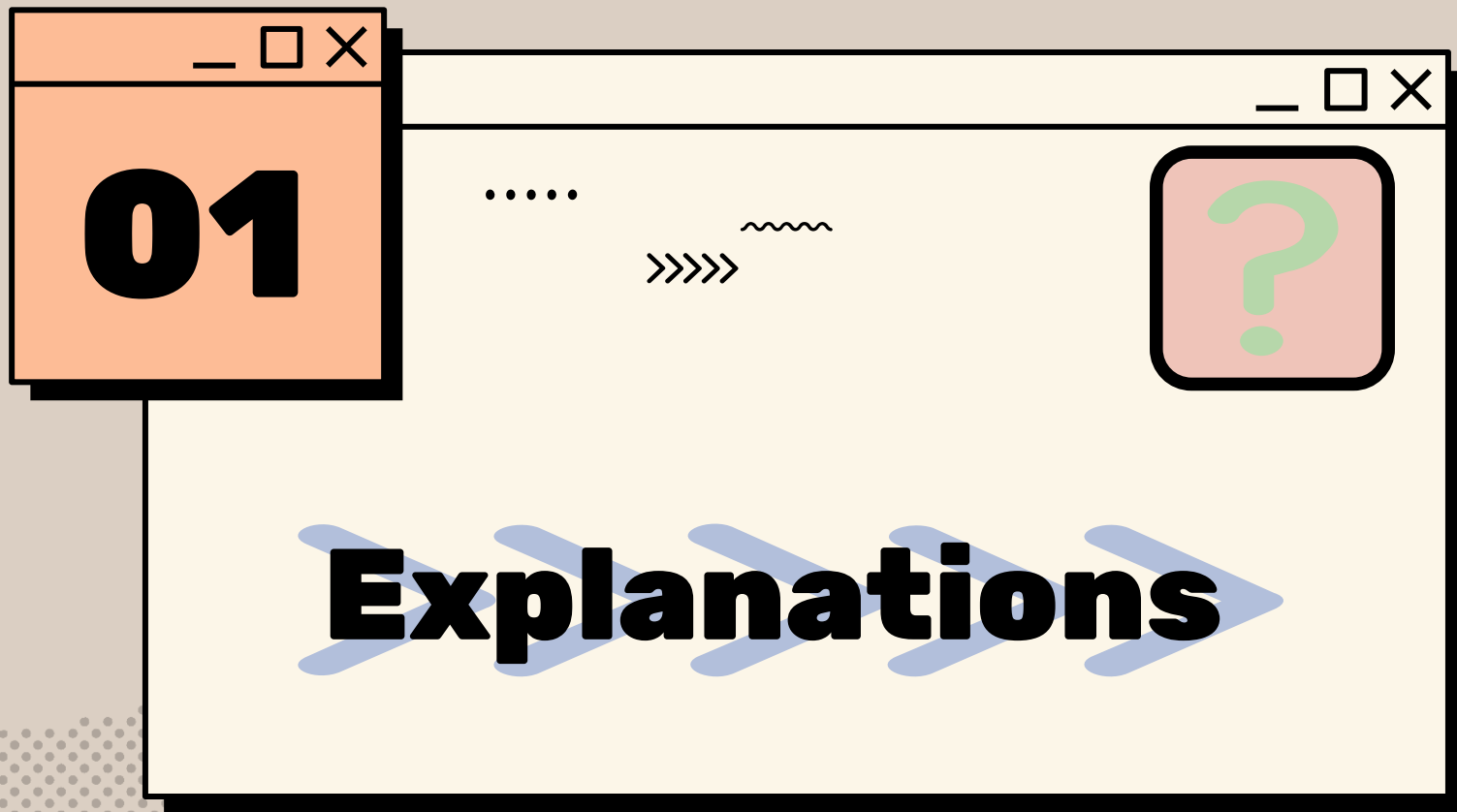


Paula Rivas González y Eloy Jiménez Medina

>>>>

.....







Code Smells



Long Method

A Long Method is a type of code smell that occurs when a method contains too many lines of code or performs too many different tasks. Instead of focusing on a single responsibility, the method becomes overloaded with multiple actions.

Large Class

A Large Class is a type of code smell that occurs when a class contains too many fields, methods, or responsibilities. Instead of representing a single, well-defined concept, the class tries to handle multiple concerns at once.

01 Bloaters

Bloaters are a type of code smell that appear when parts of the code grow too large, too complex, or overloaded with responsibilities over time. They make the code harder to understand, maintain, and modify



Code Smells



Switch Statements

The excessive use of complex switch statements or if statements to determine behaviour based on the object type, rather than using polymorphism

Temporary Field

An object has fields that are only used under certain circumstances and remain empty or null the rest of the time.

02

Object-Oriented Abusers

Object-Oriented abusers is a type of code smell that occurs when the principles of object-oriented programming are applied incorrectly, incompletely or excessively



Code Smells



03

Change Preventers

Change Preventers are code smells that occur when a single change in the system requires modifying multiple places in the code. This makes the code difficult to maintain and increases the risk of introducing bugs.

Divergent Change

Divergent Change happens when a single class or module is frequently modified for different and unrelated reasons. In other words, one class is responsible for too many functionalities, and every new requirement forces changes in the same place.

Shotgun Surgery

Shotgun Surgery is a type of code smell that occurs when a single change in the system requires modifying many different classes or files. Instead of being localized, the impact of a change is scattered across the codebase.



Code Smells



04 Dispensables

Dispensables are parts of the code that are unnecessary or provide no real value to the system. They make the code harder to read, maintain, and understand without improving functionality.

Dead Code

Dead code refers to any part of the code that is never used or executed during the program's runtime.

Duplicate Code

Duplicate code occurs when the same or very similar code is repeated in multiple places instead of being reused.



Code Smells



05 Couplers

Couplers are a category of code smells that indicate an excessive coupling between classes or modules, thereby violating the principles of object-oriented design

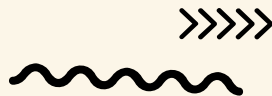
Feature Envy

A method uses data from another object rather than its own.

Inappropriate Intimacy

One class uses the private fields or methods of another class.

Comparisons



02



Before

```
// 1 DISPENSABLE: Dead Code (start)
private String temp; no usages
// unused variables
// DISPENSABLE: Dead Code (end)
```



After

1 Code Smells

This code smell is a “Dispensable” more specifically, “Dead code”. Therefore, the best solution is to remove that part of the code that we don’t use.

As you can see, the code isn't being used, which is why the “no usages” message appears; we then simply remove it.



2 Code Smells

This code smells: the names are long and difficult to distinguish. This makes them difficult to use.

The solution we have implemented to make them easier to use is to replace the names with shorter, more intuitive ones.

Before

```
//2 VERY LONG NAME  
private By personwhowantsloginintothepage = By.id("user-name"); 3 usages  
private By thatPasswordIsCorrect = By.id("password"); 3 usages  
private By thisIsALoginButton = By.id("login-button"); 2 usages  
private By thisIsAErrorMessage = By.cssSelector("[data-test='error']"); 1 usage
```



After

```
private By userField = By.id("user-name"); 1 usage  
private By passField = By.id("password"); 1 usage  
private By loginButton = By.id("login-button"); 1 usage  
private By errorMessage = By.cssSelector("[data-test='error']");
```



Before

```
// 3 DISPENSABLE: Duplicate Code (start)
public void writeUser(String user) { driver.findElement(personwhowantsloginintothepage).sendKeys(user); }
public void writeUser2(String user) { driver.findElement(personwhowantsloginintothepage).sendKeys(user); }
// DISPENSABLE: Duplicate Code (end)
```

3 Code Smells



This code smell is a “Dispensable” more specifically, “Duplicate Code”. Therefore, the best solution is to remove the duplicated section of code. In this case, we need to remove the duplicate method.

After

```
//This is so you can enter your username in the appropriate field
public void writeUser(String user) { driver.findElement(userField).sendKeys(user); }
```



Before

```
// 4 DISPENSABLE: Duplicate Code (start)
public void writePassword(String password){ 2 usages  @ejm0005
    driver.findElement(thatPasswordIsCorrect).sendKeys(password);
    driver.findElement(thatPasswordIsCorrect).sendKeys(password);
}
// DISPENSABLE: Duplicate Code (end)
```



After

```
//This is so you can enter the password in the appropriate field
public void writePassword(String password){ driver.findElement(passField).sendKeys(password); }
```

4 Code Smells

This code smell is a “Dispensable” more specifically, “Duplicate Code”. Therefore, the best solution is to remove the duplicated section of code. In this case, we need to remove the duplicate line for its method.



5 Code Smells

This code smell is a “Dispensable” more specifically, “Duplicate Code”. Therefore, the best solution is to remove the duplicated section of code.
In this case, we need to remove the duplicate line for its method.

Before

```
// 5 DISPENSABLE: Duplicate Code (start)
public void clickLogin() { driver.findElement(thisIsALoginButton).click(); // duplicate }
public void clickLogin2() { driver.findElement(thisIsALoginButton).click(); // duplicate }
// DISPENSABLE: Duplicate Code (start)
```



After

```
//To be able to click the login button
public void clickLogin() { driver.findElement(loginButton).click(); // duplicate }
```



Before



```
// 6 + 7 BLOATERS: Long Method (start) + CHANGE PREVENTER: Divergent Change (start)
public void login(String usuario, String contrasena){ no usages new *

    // 8 OBJECT-ORIENTATION ABUSER: Switch (start)
    if(usuario.equals("user")){
        escribirUsuario("user");
        escribirContrasena("1234");
    }
    // OBJECT-ORIENTATION ABUSER: Switch (end)

    escribirUsuario(usuario);
    escribirContrasena(contrasena);
    pulsarLogin();

    // 9 DISPENSABLE: Dead Code (start)
    int a = 5;
    int b = 10;
    int c = a + b;
    // unused variables
    // DISPENSABLE: Dead Code (end)

    // 10 CHANGE PREVENTER: Shotgun Surgery (simulate)
    driver.findElement(personaqueestaenmtrandoalpaguina).clear();
    driver.findElement(estacontrasefaseracorreta).clear();
    // If this changes, it will have to be updated in many places
    guardarLog();
}
// BLOATERS: Long Method (end) + CHANGE PREVENTER: Divergent Change (end)
// It does too many things in a single method
```

6+7 Code

Code smells 6 and 7 are two different errors bloater: Long Method and change preventer: Divergent Change, but they have the same solution.

The Long Method is resolved by breaking the code down into smaller parts; the issue with Divergent Change is that the code tries to do too many things. The best solution is to focus the code on a single method, so we also shorten the method.

After



```
//This method requires you to enter your login details
public void login(String user, String password){ 3 usages 8 pauri +1
    writeUser(user); //Enter user
    writePassword(password); //Enter password
    clickLogin(); //Click the button Login
}
```



8 + 9 Code

This Code smells 8 and 9 are two distinct issues: 'Object-Orientation Abusers: Switch' and 'Dispensables: Dead Code' share the same result and form part of the solution to the previous code smells.

The code 8 would be to modify the code so that it no longer contains an "if" statement, as this would no longer be necessary, and option 9 would be to remove the code.

Before

```
// 9 DISPENSABLE: Dead Code (start)
int a = 5;
int b = 10;
int c = a + b;
// unused variables
// DISPENSABLE: Dead Code (end)
```



Before

```
// 8 OBJECT-ORIENTATION ABUSER: Switch (start)
if(user.equals("user")){
    writeUser("user");
    writePassword("1234");
}
// OBJECT-ORIENTATION ABUSER: Switch (end)
```



After

```
//This method requires you to enter your login details
public void login(String user, String password){ 3 usages 2 pauri +1
    writeUser(user); //Enter user
    writePassword(password); //Enter password
    clickLogin(); //Click the button Login
}
```



10 Code Smells

Code smell 10 is a type of 'Change Preventer'; as it forms part of the previous method, it has the same effect as the previous ones.

It is also an issue we need to watch out for, because if a change is made, we'll have to update it everywhere. Since we wanted to focus on a single action within the method, we've removed this part of the code.

Before

```
// 10 CHANGE PREVENTER: Shotgun Surgery (simulate)
driver.findElement(personwhowantsloginintothepage).clear();
driver.findElement(thatPasswordIsCorrect).clear();
// If this changes, it will have to be updated in many places
```



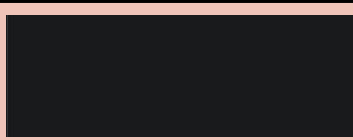
After

```
//This method requires you to enter your login details
public void login(String user, String password){ 3 usages 2 pauri +1
    writeUser(user); //Enter user
    writePassword(password); //Enter password
    clickLogin(); //Click the button Login
}
```

Code Smell 11 is an unnecessary method called `saveLog()`. The best solution is to remove it, because it is irrelevant to the functional logic, and replace its invocations with the `login()` method.

11 Code Smells

After



Before

```
saveLog();  
}
```

Before

```
// 11 Unnecessary method added  
public void saveLog(){ 1 usage  
    System.out.println("Login");  
}
```

After

```
//This method requires you to enter your login details  
public void login(String user, String password){ 3 usages 2 pauri +1  
    writeUser(user); //Enter user  
    writePassword(password); //Enter password  
    clickLogin(); //Click the button Login  
}
```

Before

```
// 12 COUPLERS: Feature Envy (start)
public boolean validateUserFormat(UserSession session) { no usages
    return session.username.contains("@") && session.username.length() > 5 && !session.username.isEmpty();
}
// COUPLERS: Feature Envy (end)
```

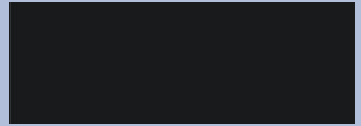
Before

```
// Class for Couplers
class UserSession { 2 usages  @ejm0005
    public String username; 5 usages
    private String _internalToken = "SECRET_123"; no usages

    public String getUsername() { return username; } no usages @ejm0005
}
```



After



Code Smell 12: Feature Envy. LoginPage is overly coupled with UserSession. Given that UserSession provides no unique functional value and results in redundant logic, the best solution is to delete the class and consolidate the necessary behavior within LoginPage.

12 Code Smells

Before

```
// 13 COUPLERS: Inappropriate Intimacy (start)
public void hackSessionToken(UserSession session) { no usages
    System.out.println("Accessing data I'm not authorized to see: " + session.username);
}
```

Before

```
// Class for Couplers
class UserSession { 2 usages  @ejm0005
    public String username; 5 usages
    private String _internalToken = "SECRET_123"; no usages

    public String getUsername() { return username; } no usages  @ejm0005
}
```



After



Code Smell 13 is a Inappropriate Intimacy. The hackSessionToken method attempts to access private elements of the UserSession class without authorization. As with Code Smell 12, this method provides no unique functional value and results in redundant logic; therefore, the best solution is to remove it.

13 Code Smells



Refactoring summary



This code contains many code smells, so we have applied several refactoring techniques. These include:

- Extract Method to reduce the length of methods.
- Remove Duplicate Code to eliminate repetition.
- Remove Dead Code to clean up unused elements.
- Remove a Class to avoid mixing responsibilities.

With all these changes in place, we are left with cleaner, more functional code.





Final Reflection

>>>>



This project has been a great help to us because we've learnt how to work with branches using IntelliJ IDEA and GitHub. It has also helped us to consolidate the Maven concepts we learnt in previous projects. Overall, it has helped us to see what it would be like to put this into practice in a work environment.

....



